

mCOLD BLE Protocol (สำหรับทีมแอป)

เวอร์ชัน 0.3 (ฉบับร่าง) · 2026-10-07 · firmware 0.7.0-dev.47

สถานะ: เสนอ (protocol 1) ปรับปรุง 7 ต.ค. 2026 เขียนโดยฝั่ง firmware ให้ทีมแอปปรับหรือขอแก้ ยังไม่ถือว่าสุดท้ายจนกว่าทีมแอปจะรับ โดยเฉพาะ UUID ที่จะตรงเมื่อแอปทำตามแล้ว

เอกสารนี้สำหรับทีมแอป ส่วนที่ส่งคุยกับ server ผ่าน MQTT อยู่ใน [../server/](../server/README.md) JSON คำขอและคำตอบชุดเดียวกันใช้ได้กับทุกช่องทาง ตอนนี้ใช้ทาง BLE และ USB serial (คำสั่ง rpc ใน console สำหรับทดสอบ) ภายหลังจะเพิ่มช่องทางอื่นโดยไม่ต้องเปลี่ยนรูปแบบ

สิ่งที่เปลี่ยนจากเอกสารรอบแรก (0.1)

เรื่อง	เดิม	ตอนนี้
ชื่อ trip	trip (เลขนับในกล่อง เช่น 14)	trip_id (UUID ที่กล่องสร้างเอง) + trip_date (YYYYMMDD) + trip_number (เลขรันของวัน)
คำสั่ง BLE ที่อ้าง trip	"trip": 14	"trip_id": "<uuid>" (MARK_DELIVERED, GET_TRIP_SUMMARY, READ_LOG_CHUNK)
คำตอบที่บอก trip	"trip": 14	trip_id, trip_date, trip_number
คอลัมน์ในแถวและ CSV	trip, seq, sn, ...	trip_id, trip_date, trip_number, seq, ... (MQTT ไม่มี sn, utc, detail; timestamp เป็น Unix)
รายการ Wi-Fi	ส่งเฉพาะที่มี เรียงตามชื่อ	ส่งครบ 5 ช่อง (slot 1–5, ช่องว่าง ssid: null) แยกหรือลบช่องไหนก็ได้ด้วยเลขช่อง
trip เก่า (ก่อน dev.34)	ส่งขึ้น server ได้	ไม่ส่งและไม่ปรากฏใน LIST_TRIPS เพราะไม่มี trip_id

เลขนับในกล่องยังมีอยู่ข้างใน แต่ไม่ออกมาข้างนอกอีก

1. หากกล่อง

1. แอปอ่านแท็ก NFC เป็น NDEF record ชนิด MIME application/json :

```
{ "sn": "mCDV1-L0169-1069-002", "ble": "28:84:85:27:9B:86", "key": "571ab30e58225355ccfc9b010a935f65" }
```

sn และ ble มีตั้งแต่ช่วง bring-up ส่วน key คือรหัสยืนยันตัวปัจจุบันของกล่อง (ดูหัวข้อ Authorization ในข้อ 2) กล่องปรับข้อมูลในแท็กเอง โดยเขียนเฉพาะไบต์ที่เปลี่ยน ทีมแอปเป็นเจ้าของรูปแบบนี้ firmware จะเพิ่ม URI record (สำหรับแบนเนอร์บน iOS) เมื่อมี URL ให้ใส่

- 1. การแตะและปลุก BLE ของกล่องด้วย: กล่องประกาศตัว (advertise) 60 วินาที และ**หลังแอปตัดการเชื่อมต่อ กล่องประกาศต่ออีก 5 วินาทีแล้วปิด BLE** (ตอนเสียบไฟ USB ก็เช่นกัน) ถ้าจะเชื่อมใหม่หลังจากนั้นต้องแตะ NFC อีกครั้ง key ยังใช้ได้ภายใน 2 นาทีหลังจบ session จึงไม่ต้องอ่านค่าใหม่ถ้าอ่านจากการแตะรอบล่าสุด
- 2. **iOS ไม่เห็น BLE MAC** แอปหากกล่องจากชื่อที่ประกาศ ซึ่งคือ SN และกรองด้วย service UUID ส่วน MAC ใน NDEF ใช้ตัดสินเมื่อมีหลายตัวเท่านั้น ตามที่ทีมแอปยืนยัน

การประกาศ: flags + service UUID แบบ 128 บิตใน advertisement, ชื่อเต็ม (SN) ใน scan response, ช่วง 20–40 ms ระหว่างที่หน้าต่างเปิด

2. BLE GATT

มี primary service เดียว ทุก UUID ใช้ฐาน d2a5xxxx-3818-4667-a205-b3ed9c37b8a5

UUID	ชื่อ	คุณสมบัติ	ความปลอดภัย
d2a50000-...	mCOLD service		
d2a50001-...	INFO	read	เปิด
d2a50002-...	STATUS	read, notify	เปิด
d2a50003-...	COMMAND	write	เปิด แต่สิ่งที่ส่งได้ขึ้นกับ AUTH
d2a50004-...	RESPONSE	notify	เปิด
d2a50005-...	EVENT	notify	เปิด

- **INFO** คือผลของ GET_INFO (ข้อ 4) แอปเช็คเวอร์ชัน protocol ได้ก่อนส่งอะไร
- **STATUS** คือผลของ GET_STATUS แจ้ง (notify) เมื่อสถานะ trip หรือ alarm เปลี่ยน และหลังบันทึกแต่ละครั้ง
- **COMMAND** รับคำขอ **RESPONSE** ส่งคำตอบกลับ
- **EVENT** แจ้งเหตุการณ์ของ trip และ alarm: { "ev": "ALARM_RAISE", "alarm": "TEMP_HIGH" }, ALARM_CLEAR, ALARM_ACK, TRIP_START, TRIP_STOP และ **WIFI** (สถานะ Wi-Fi เปลี่ยน ดูหัวข้อ "แอปเห็นค่าที่เปลี่ยนในกล่องอย่างไร")
- **SETTINGS** มากับ EVENT เอง (ตกลง 5 ต.ค.): กันที่ทีมแอป subscribe, อีกครั้งเมื่อ session ผ่าน AUTH และเมื่อมีการตั้งค่าเปลี่ยน (จากแอป server หรือ console) ทำให้หน้าตั้งค่าของแอปตรงกับกล่องเสมอ: { "ev": "SETTINGS", "config": { ...ทุกค่า... }, "editable": [...], "trip_locked": [...], "server_rev": 1, "network": { ... } } ส่วน network (Wi-Fi แต่ละเครือข่ายพร้อม DHCP หรือ IP คงที่ และ OTA base ไม่มีรหัสผ่าน) จะมีหลัง AUTH เท่านั้น ขนาดราว 1.1 KB จึงมาเป็นหลาย fragment

Authorization: แตะเพื่อยืนยันตัว

ตกลง 2 ต.ค. แทนการพิมพ์รหัสบนจอ **ไม่ต้อง pair BLE** มือถือไม่ขึ้นหน้าต่างของระบบ และไม่ต้องพิมพ์อะไร

ใครอยู่ในระยะ BLE ก็อ่าน INFO และ STATUS ได้ ส่วนคำสั่งที่เปลี่ยนค่า (มีเครื่องหมาย 🗑 ในข้อ 4) ต้องมี session ที่ยืนยันตัวแล้ว ซึ่งยืนยันได้ด้วยการพิสูจน์ว่าแอปอ่านแท็ก NFC มาซึ่งอ่านได้ในระยะไม่ที่เซนติเมตร จึงต้องถือกล่องอยู่

1. แตะ: อ่าน key (16 ไบต์ เป็น hex) จาก record ของแท็ก
2. เชื่อม BLE และอ่าน INFO ในนั้นมี nonce เลขสุ่ม 16 ไบต์ (hex) ใหม่ทุกการเชื่อมต่อ
3. ส่ง { "id": ..., "cmd": "AUTH", "proof": "<hex>" } โดย proof = HMAC-SHA256(key, nonce) คำนวณจาก**ไบต์ดิบ**ของทั้งสองค่า (ไม่ใช่ข้อความ hex) และส่งผลเป็น hex 64 ไบต์
4. { "ok": true } คือการเชื่อมต่อนี้สั่งงานได้จนกว่าจะปิด ถ้าได้ NOT_AUTHORIZED ให้แตะใหม่แล้วลองอีก

ข้อควรระวัง (จากการทดสอบแอปวันที่ 7 ต.ค.): ช่องที่ AUTH อ่านคือ proof ไม่ใช่ key ถ้าส่ง {"cmd": "AUTH", "key": "..."} กล้องจะตอบ NOT_AUTHORIZED เสมอ และห้ามส่ง key ทาง BLE ไม่ว่ากรณีใด

key ไม่เคยเดินทางผ่าน BLE คนดักสัญญาณเห็นแค่ nonce กับ proof ซึ่งใช้กับการเชื่อมต่อกันไม่ได้

key เปลี่ยนเมื่อกล้องรีเซ็ต และ 2 นาทีหลังจบ session ที่ใช้ key นั้น และหนึ่งครั้งได้หนึ่ง session (ถ้าลิงก์หลุดกลางทาง เชื่อมกลับภายใน 2 นาทีด้วยการแตะเดิมได้) กล้องเขียน key ใหม่ลงแฟล็กก่อนเริ่มรับ แฟล็กจึงไม่เคยมี key ที่กล้องจะปฏิเสธ การแตะ NFC เฉยๆ ไม่ทำให้ key เปลี่ยน

ตัวอย่างจาก test client ของ firmware:

```
key 7b3066da5096846781a0b5c1d0f6699b
nonce 951c0f73... (จาก INFO)
proof = HMAC-SHA256(key, nonce) -> {"cmd": "AUTH", "proof": "..."} -> ok
```

Fragments (การแบ่งข้อความ)

ข้อความยาวกว่าการเขียนหรือการ notify ครั้งเดียว ทั้งสองทิศจึงใช้กรอบเดียวกันโดยไม่ขึ้นกับ MTU:

```
byte 0 flags: bit 7 FIRST, bit 6 LAST, bits 0-5 ลำดับ fragment (mod 64)
byte 1.. ส่วนหนึ่งของ JSON แบบ UTF-8
```

ข้อความหนึ่งคือ fragment FIRST ถึง LAST ตามลำดับ (BLE write with response และ notify เรียงลำดับอยู่แล้ว) ข้อความที่มี fragment เดียวตั้งทั้งสองบิต fragment ที่ลำดับผิด หรือรวมเกิน 4,096 ไบต์ จะทิ้งทั้งข้อความ และกล้องตอบ BAD_REQUEST ด้วย id 0 คำตอบถูกตัดให้พอดี MTU ที่ตกลงกัน (MTU - 3 ไบต์ของ ATT - 1 ไบต์ของ flags)

3. รูปแบบข้อความ

คำขอ:

```
{"id": 17, "cmd": "START_TRIP", "low": 2.0, "high": 8.0}
```

คำตอบ:

```
{"id": 17, "ok": true, "trip_id": "95518e06-1b3b-494f-b4c5-284ae94f5424", "trip_date": "20261007", "trip_number": 2}
{"id": 17, "ok": false, "err": "ALREADY_ACTIVE", "msg": "a trip is already running"}
```

- id: แอปเลือก ค่า 1 ถึง 4294967295 ไม่ซ้ำกันต่อคำขอ คำตอบส่ง id กลับมา
- Idempotency:** ทุกคำสั่งที่เปลี่ยนค่า (มี ✎) จำ id ของตัวเอง ส่ง id เดิมอีกครั้งจะได้คำตอบเดิมโดยไม่ซ้ำซ้ำ ดังนั้นเมื่อหมดเวลาหรือลิงก์หลุด แอปลองใหม่ด้วย id เดิม ไม่ใช่ id ใหม่ กล้องจำ 8 รายการล่าสุดใน RAM ส่วน START_TRIP จำใน flash ด้วย ลองซ้ำหลังกล้องรีเซ็ตก็ได้ trip เดิม **อย่าใช้ id ซ้ำกับคำสั่งคนละอัน** เพราะจะได้คำตอบของคำสั่งเก่า
- ช่องที่กล้องไม่รู้จักในคำขอจะถูกข้าม เพิ่มช่องได้
- ส่ง proto ได้ ถ้าเวอร์ชันที่กล้องไม่รองรับจะตอบ UNSUPPORTED_PROTO

รหัส error:

err	ความหมาย
BAD_REQUEST	ไม่ใช่ JSON, ไม่มี id หรือไม่มี cmd
UNKNOWN_CMD	กล้องไม่รู้จักรหัส cmd (ต้องเป็นตัวพิมพ์ใหญ่ตามข้อ 4 เช่น APPLY_CONFIG ไม่ใช่ set)
BAD_ARGS	ขาดหรือเกินช่วงของ argument
NOT_AUTHORIZED	ต้องผ่าน AUTH ก่อน หรือ AUTH ไม่ผ่าน
UNSUPPORTED_PROTO	proto ใหม่เกินไป
ALREADY_ACTIVE / NOT_ACTIVE	สถานะ trip ไม่ให้ทำ
LOG_FULL , NO_LOG , FLASH	พื้นที่เก็บค่าไม่ได้
NOT_SUPPORTED	กำหนดไว้แล้ว แต่ firmware นี้ยังไม่มี
EXISTS	เพิ่มแล้วซ้ำ: เครือข่ายนั้นมีอยู่แล้ว หรือช่องนั้นมีเครือข่ายอยู่ (ADD_WIFI)
NOT_FOUND	ช่องที่ระบุว่างอยู่ หรือไม่มีเครือข่ายชื่อนั้น (EDIT_WIFI , DEL_WIFI)
FULL	ครบ 5 ช่องแล้ว (ADD_WIFI)

หน่วย: อุณหภูมิเป็น °C ตัวเลข, เวลาเป็นวินาที Unix UTC พร้อมคุณภาพแยกต่างหาก (none , rtc , gnss , host , ntp), ระยะเวลาเป็นวินาที ค่าที่กล้องไม่มีจะ**ไม่ใช่หรือเป็น null** **ไม่เคยเป็น 0**

ตัวตนของ trip (ตกลง 7 ต.ค.)

ช่อง	ตัวอย่าง	ความหมาย
trip_id	95518e06-1b3b-494f-b4c5-284ae94f5424	UUID (v4) ที่กล้องสร้างตอนเริ่ม trip ทำให้ server ผูก trip กับ sn ได้โดยไม่ซ้ำกัน
trip_date	"20261007"	วันที่ท้องถิ่นตอนเริ่ม trip รูปแบบ YYYYMMDD เป็นข้อความ (ถ้ากล้องไม่รู้เวลาตอนเริ่มจะเป็น "00000000")
trip_number	2	เลขรันของวันนั้น เริ่มที่ 1 และเริ่มใหม่เมื่อวันเปลี่ยน

ใช้ trip_id เป็นตัวอ้างอิงในทุกคำสั่งและ ACK ส่วน trip_date กับ trip_number ไว้ให้คนอ่าน เช่น "trip ที่ 2 ของวันที่ 7 ต.ค."

4. คำสั่ง

คำสั่ง	Argument	ผลลัพธ์
GET_INFO		proto, sn, fw, hw, idf, boot, uptime_s; ภาวะ BLE มี nonce และ authorized ด้วย
AUTH	proof (hex)	ยืนยันตัวการเชื่อมต่อนี้ (ข้อ 2)
GET_STATUS		ดูด้านล่าง
GET_CONFIG		config: ทุกค่าพร้อมค่าปัจจุบัน
SET_CONFIG	key, value	ตั้งค่าหนึ่งตัว ทาง BLE ตั้งได้เฉพาะ key ใน device-settings.md ค่าที่ ลือระหว่าง trip ตั้งไม่ได้ขณะ trip วิ่ง (key ในที่นี้คือ ชื่อค่าที่ตั้ง ไม่ใช่รหัสจาก NFC)
SET_TIME	utc (Unix วินาที)	quality
START_TRIP	low, high (°C); hyst (°C, 0.5), dwell_s (300)	trip_id, trip_date, trip_number
STOP_TRIP		trip_id, trip_date, trip_number
ACK_ALARM		alarms ที่ยังคงค้าง
LIST_TRIPS		trips: [{"trip_id", "trip_date", "trip_number", "last_seq", "sent"}] เรียงเก่าก่อน; sent คือ server ได้ถูกแถวแล้ว; trip ที่ไม่มี trip_id ไม่อยู่ในรายการ
APPLY_CONFIG	config, wifi (add: แต่ละรายการมี slot ได้, del: เลขช่องหรือชื่อ), ota_base (ใส่ตัวไหนก็ได้; mqtt ปฏิเสธ: ตั้งทาง console เท่านั้น)	applied, errors: เอกสารการตั้งค่าตาม device-settings.md ชุดเดียวกับที่ server ส่งที่ mcold/v1/<sn>/config เฉพาะ key ที่ระบุไว้
GET_NETWORK		wifi: 5 ช่องเสมอ [{"slot":1,"ssid":"...", "dhcp":true[, "ip", "gateway", "subnet", "dns"]}, {"slot":2,"ssid":null[, ...] , ota_base ไม่มีรหัสผ่าน ต้อง AUTH (ดูหัวข้อ Wi-Fi 5 ช่อง)
GET_SETTINGS		เนื้อหาของ event SETTINGS แบบตนเอง; network (Wi-Fi 5 ช่องแบบเดียวกับ GET_NETWORK) มีหลัง AUTH
MARK_DELIVERED	trip_id	trip_id, trip_date, trip_number, rows: แอปส่ง trip ที่จบแล้วให้ server เองแล้ว ถล่องจะ ไม่อัปเดตซ้ำ (ได้ ALREADY_ACTIVE ถ้า trip ยังวิ่ง)
GET_TRIP_SUMMARY	trip_id	trip_id และเกณฑ์, samples, min, max, alarms, stopped
READ_LOG_CHUNK	trip_id, from (seq), max (1-16)	records, next
GET_STORAGE_STATUS		sectors, used, free, trips, days_left
SELF_TEST		health: อุปกรณ์แต่ละตัวและสถานะ
REBOOT		(ตอบก่อน แล้วถล่องรีสตาร์ท)
GET_SYNC_STATUS		wifi (เชื่อมไหม, ssid, rssi, known: ชื่อที่รู้จัก), server (broker, pending แถวค้าง, last_ack_s)
SYNC_NOW		อัปเดตเลยแทนที่จะรอรอบถัดไป
ADD_WIFI	ssid, pass (ว่าง "" สำหรับเครือข่ายเปิด, dhcp หรือ ip, gateway, subnet, dns, slot (1-5 ไม่ใส่ก็ได้)	เพิ่ม เครือข่ายใหม่ลงช่อง slot หรือช่องว่างช่องแรก ได้ slot ที่ใช้และรายการ Wi-Fi ทั้ง 5 ช่อง EXISTS ถ้าชื่อนี้มีอยู่แล้วหรือช่องไม่ว่าง, FULL ถ้าครบ 5 ช่อง
EDIT_WIFI	slot (1-5 จำเป็น), ssid, pass, dhcp หรือ ip, gateway, subnet, dns (ใส่เฉพาะที่จะเปลี่ยน)	แก้ ช่องที่มีเครือข่ายอยู่ที่ไม่ส่งมายังคงเดิม เปลี่ยน ssid ต้องส่ง pass ด้วย (และกลับเป็น DHCP จนกว่าจะส่ง ip) ได้รายการทั้ง 5 ช่อง NOT_FOUND ถ้าช่องว่าง
DEL_WIFI	slot (1-5) หรือ ssid	ลบ เครือข่าย ช่องนั้นว่าง ช่องอื่นไม่ยับ ได้รายการทั้ง 5 ช่อง NOT_FOUND ถ้าช่องว่างหรือไม่มีชื่อนั้น
SET_WIFI	เหมือน ADD_WIFI / EDIT_WIFI รวมกัน	แบบเก่า: เพิ่มถ้าไม่มี แต่ถ้ามีแล้ว (ใช้ต่อได้ แต่ควรใช้ 3 คำสั่งข้างบนที่ชัดเจนกว่า)
GET_USB_SNAPSHOT_STATUS		NOT_SUPPORTED จนกว่าจะทำเรื่อง USB drive

Wi-Fi 5 ช่อง (ตกลง 7 ต.ค.)

ถล่องจำ Wi-Fi ได้ 5 ช่อง ตำแหน่งคงที่ ช่องที่ลบแล้วเป็นช่องว่างและช่องอื่นไม่ยับ แอปจึงอ้าง "ช่อง 3" แล้วได้เครือข่ายเดิมเสมอจนกว่าจะแก้ GET_NETWORK (และ network ใน event SETTINGS) ส่งครบทั้ง 5 ช่อง:

```
{ "id":10, "ok":true, "wifi": [
  { "slot":1, "ssid": "RDE_2.4GHz", "dhcp":true },
  { "slot":2, "ssid": null },
  { "slot":3, "ssid": "Warehouse", "dhcp":false, "ip": "192.168.1.50", "gateway": "192.168.1.1", "subnet": "255.255.255.0", "dns": "" },
  { "slot":4, "ssid": null },
  { "slot":5, "ssid": null } ], "ota_base": null }
```

แอปเพิ่ม แก่ ลบ ช่องโหนดได้ด้วย 3 คำสั่งที่แยกกันชัดเจน แต่ละคำสั่งตอบรายการทั้ง 5 ช่องกลับมาให้ แอปอัปเดตหน้าได้เลยโดยไม่ต้องอ่านซ้ำ (และกล่องส่ง event SETTINGS ตามให้ด้วย):

ผู้ใช้ทำ	คำสั่ง	ตัวอย่าง
เพิ่ม เครือข่ายใหม่	ADD_WIFI	{ "cmd": "ADD_WIFI", "ssid": "Shop", "pass": "รหัส", "dhcp": true } ลงช่องว่างช่องแรก หรือใส่ "slot": 2 ให้ช่องที่เลือก
เพิ่ม พร้อม IP คงที่	ADD_WIFI	{ "cmd": "ADD_WIFI", "ssid": "Shop", "pass": "รหัส", "dhcp": false, "ip": "10.1.1.5", "gateway": "10.1.1.1", "subnet": "255.255.255.0" }
เปลี่ยน IP หรือ DHCP ของช่อง	EDIT_WIFI	{ "cmd": "EDIT_WIFI", "slot": 3, "dhcp": true } ไม่ต้องส่ง ssid / pass
เปลี่ยน รหัสผ่าน ของช่อง	EDIT_WIFI	{ "cmd": "EDIT_WIFI", "slot": 3, "pass": "รหัสใหม่" }
เปลี่ยน ชื่อเครือข่ายใน ช่อง	EDIT_WIFI	{ "cmd": "EDIT_WIFI", "slot": 3, "ssid": "ชื่อใหม่", "pass": "รหัส" } ต้องมี pass เสมอ
ลบช่อง	DEL_WIFI	{ "cmd": "DEL_WIFI", "slot": 3 } (หรือ "ssid": "ชื่อ")

คำตอบของทั้งสามคำสั่ง (ตัวอย่างของ ADD_WIFI):

```
{ "id": 21, "ok": true, "slot": 4, "wifi": [ { "slot": 1, "ssid": "RDE_2.4GHz", "dhcp": true }, { "slot": 2, "ssid": null }, { "slot": 3, "ssid": "Warehouse", "dhcp": true }, { "slot": 4, "ssid": "Shop", "dhcp": true }, { "slot": 5, "ssid": null } ], "ota_base": null }
{ "id": 22, "ok": false, "err": "EXISTS", "msg": "already in the list: EDIT_WIFI it" }
```

APPLY_CONFIG ที่มี wifi.add / wifi.del ยังใช้ได้ (ตั้งหลายค่าพร้อมกันกับค่าอื่น แต่ add ทำหน้าที่ทั้งเพิ่มและแก้ไข ไม่แยกว่าเพิ่มซ้ำ) ถ้าแอปทำหน้ารายการ Wi-Fi แนะนำให้ใช้ 3 คำสั่งข้างบน

- ไม่ส่งรหัสผ่านกลับมา: แอปจึงดูรหัสเดิมไม่ได้ ถ้าไม่เปลี่ยนรหัส ก็ไม่ต้องส่ง pass
- ห้ามซ้ำ: เครือข่ายเดียวกันอยู่ได้ช่องเดียว ADD_WIFI ได้ EXISTS; EDIT_WIFI ที่เปลี่ยนชื่อไปซ้ำกับอีกช่องได้ BAD_ARGS (that network is already in another slot)
- ผลลัพธ์รายการ: เฉพาะ APPLY_CONFIG คำตอบมี applied และ errors แยกรายช่อง ส่วน 3 คำสั่งใหม่ทำที่ละช่อง สำเร็จหรือได้ error รหัสเดียว
- ส่ง slot ที่เกิน 1-5 ได้ BAD_ARGS (slot 1..5) ลบหรือแก้ไขช่องที่ว่างอยู่ได้ NOT_FOUND
- กล่องเชื่อมเครือข่ายที่สัญญาณแรงสุดในระยะ: ไม่มีลำดับความสำคัญตามช่อง

แอปเห็นค่าที่เปลี่ยนในกล่องอย่างไร

กล่องส่งข้อมูลใหม่ให้เอง แอปไม่ต้องอ่านซ้ำ:

ข้อมูล	กล่องส่งผ่าน	เมื่อไร
ค่าตั้งทั้งหมด และ network (Wi-Fi 5 ช่อง, ota_base)	event SETTINGS ที่ช่อง EVENT	ทันทีที่แอป subscribe,ทันทีที่ session ผ่าน AUTH (เพราะ network มีหลัง AUTH เท่านั้น) และทุกครั้งที่ค่าตั้งเปลี่ยน
อุณหภูมิ แบต สถานะ trip และ alarm	STATUS (notify)	เมื่อ trip เริ่มหรือจบ, alarm เปลี่ยน, ได้รับทราบ alarm และหลังบันทึกแต่ละครั้ง
trip, alarm	event TRIP_START, TRIP_STOP, ALARM_RAISE, ALARM_CLEAR, ALARM_ACK	ทันทีที่เกิด
ข้อมูลเครือข่าย Wi-Fi ที่กล่องต่ออยู่ (ชื่อ, IP, gateway, subnet, DNS, MAC)	event WIFI ที่ช่อง EVENT	ทันทีที่แอป subscribe (บอกสถานะตอนนั้นเลย ไม่ว่าต่ออยู่แล้วหรือยัง) และทุกครั้งที่กล่องต่อ Wi-Fi ใหม่ หลุด หรือได้ IP ใหม่

ค่าตั้งเปลี่ยนจากที่โหนดได้ ทั้งแอป server (เอกสาร config ทาง MQTT) และ console ของกล่อง SETTINGS ส่งให้ทุกคน

ข้อควรทำในแอป:

- ต้อง subscribe ช่อง EVENT และ STATUS จึงจะได้ข้อความเหล่านี้ ถ้าไม่ได้เชื่อมตอนค่าเปลี่ยน แอปจะได้ค่าล่าสุดทันทีที่เชื่อมแล้ว subscribe
- event เป็นแบบ notify ไม่รับประกันว่าได้ครบ ถ้าแอปหลุดชั่วคราวหรือไม่ใส่ใจ ให้เรียก GET_SETTINGS ขอค่าล่าสุด
- SETTINGS ยาว 1-2 KB มาเป็นหลาย fragment แอปต้องต่อให้ครบ (มีที่ FIRST/LAST) ก่อนอ่านเป็น JSON
- สถานะ Wi-Fi: ดูหัวข้อ "event WIFI" ด้านล่าง ถ้าแอปต้องการความแรงสัญญาณสดหรือจำนวนแถวที่ค้างส่ง ให้เรียก GET_SYNC_STATUS เป็นระยะ (อ่านได้โดยไม่ต้อง AUTH)

event WIFI (ข้อมูลเครือข่ายที่กล่องต่ออยู่)

กล่องส่งข้อมูลเครือข่ายให้แอปเองเป็น event ที่ช่อง EVENT ใน 2 กรณี:

- แอป subscribe ช่อง EVENT: ส่งสถานะตอนนั้นทันที ถ้ากล่องต่อ Wi-Fi อยู่แล้วก็ได้ข้อมูลเครือข่ายเต็ม ถ้ายังไม่ต่อก็ได้ connected: false
- สถานะเปลี่ยน: กล่องต่อ Wi-Fi ใหม่ (เครือข่ายเดิมหรือใหม่), หลุด, หรือได้ IP ใหม่
- ต่อไม่ติด: ทุกครั้งที่กล่องต่อแล้วไม่สำเร็จ (รหัสผ่านผิด หรือหาเครือข่ายไม่เจอ) กล่องส่ง event พร้อมสาเหตุ

ตัวอย่าง:

```
{ "ev": "WIFI", "connected": true, "mac": "28:84:85:27:9B:84",
  "ssid": "RDE_2.4GHz", "dhcp": true,
  "ip": "192.168.0.13", "gateway": "192.168.0.1", "subnet": "255.255.255.0", "dns": "192.168.0.1",
  "rssi": -51 }
```

หลุดหรือยังไม่ต่อ (ไม่มี `ssid`, `ip` และช่องที่เกี่ยวกับเครือข่าย):

```
{ "ev": "WIFI", "connected": false, "mac": "28:84:85:27:9B:84" }
```

ต่อไม่ติด (กล่องลองต่อใหม่เรื่อยๆ ตามรอบ ทุกครั้งที่พลาดส่ง event หนึ่งฉบับ `attempts` นับครั้งที่ติดต่อกัน):

```
{ "ev": "WIFI", "connected": false, "mac": "28:84:85:27:9B:84",
  "error": "wrong_password", "reason": 15, "ssid": "RDE_2.4GHz", "attempts": 3 }
```

ช่อง	ความหมาย
<code>connected</code>	ต่อเครือข่ายและได้ IP แล้วหรือยัง
<code>mac</code>	MAC ของ Wi-Fi ในกล่อง (ส่งเสมอ)
<code>ssid</code>	ชื่อเครือข่ายที่ต่ออยู่
<code>dhcp</code>	<code>true</code> = เครือข่ายให้ IP, <code>false</code> = กล่องใช้ IP คงที่ที่ตั้งไว้
<code>ip</code> , <code>gateway</code> , <code>subnet</code> , <code>dns</code>	ที่อยู่ที่กล่องใช้อยู่จริง
<code>rssi</code>	ความแรงสัญญาณ dBm ตอนที่ส่ง
<code>error</code>	(เฉพาะต่อไม่ติด) <code>wrong_password</code> = รหัสผ่านผิด, <code>not_found</code> = ไม่พบเครือข่ายที่จำไว้ในระยะ, <code>failed</code> = สาเหตุอื่น
<code>reason</code>	(เฉพาะต่อไม่ติด) รหัสสาเหตุจากชิป: 15, 204, 202 = รหัสผ่าน, 201 = ไม่พบเครือข่าย
<code>ssid (ตอนต่อไม่ติด)</code>	เครือข่ายที่ลองต่อ <code>null</code> เมื่อไม่พบเครือข่ายที่จำไว้เลย
<code>attempts</code>	(เฉพาะต่อไม่ติด) จำนวนครั้งที่ลองต่อไม่ติดติดต่อกัน เริ่มนับใหม่เมื่อต่อได้

- rssi** ไม่ใช่เหตุให้ส่ง: ค่าแกว่งตลอดเวลา จึงส่งมาด้วยเฉพาะตอนที่ event ถูกส่ง
- เปลี่ยนค่าในช่อง Wi-Fi อย่างเดียวไม่ส่ง WIFI**: event นี้บอกสิ่งที่กล่องต่ออยู่จริง การเปลี่ยนรายการที่จำไว้ (เพิ่ม ลบ แก้) แจ้งทาง `SETTINGS` แต่ถ้าเก่านั้นทำให้ต้องต่อใหม่ จะตามด้วย `WIFI` เมื่อต่อเสร็จ
- ต่อได้แล้ว** event `connected: true` มาแทน และ `attempts` หายไป
- ถี่แค่ไหน**: กล่องลองต่อทุก 2, 4, 8 ... สูงสุด 60 วินาที event ต่อไม่ติดจึงมาตามรอบนั้น แอปควรแสดงข้อความครั้งเดียว (เช่น "รหัสผ่านผิด") ไม่ต้องขึ้นซ้ำทุกฉบับ
- ต้อง subscribe ช่อง EVENT** (ตัวเดียวกับที่รับ `SETTINGS`) ไม่ต้อง AUTH

GET_STATUS :

```
{ "id": 3, "ok": true,
  "time": { "utc": 1791357844, "quality": "ntp", "boot": 42 },
  "temp": { "ok": true, "c": 4.25 },
  "trip": { "active": true,
    "trip_id": "95518e06-1b3b-494f-b4c5-284ae94f5424",
    "trip_date": "20261007", "trip_number": 2,
    "samples": 12, "min": 3.5, "max": 5.25,
    "alarms": [ "TEMP_HIGH" ], "acked": false,
    "alarms_raised": 1, "last_alarm": { "type": "HIGH", "utc": 1791358100 },
    "power": { "soc": 78, "mv": 3987, "ma": -12, "charge": "none", "external": false },
    "gnss": { "fix": false, "lat": null, "lon": null, "age_s": null },
    "storage": { "used_pct": 1 },
    "fw": { "ver": "0.7.0-dev.35", "slot": "ota_0" },
    "net": { "connected": true, "ssid": "Office", "rssi": -59, "ip": "192.168.1.147",
      "mac": "28:84:85:27:9A:74" } }
```

`trip` ใน `STATUS` เมื่อไม่มี `trip` จึงจะมี `trip_id` ของ `trip` ล่าสุดที่จบ (`active` เป็น `false`) หรือ `null` ทั้งสามช่องถ้ายังไม่เคยมี `fw` คือ firmware ที่รันอยู่และ `slot OTA` ที่รัน; `net.ssid`, `rssi` และ `ip` มีเฉพาะตอนเชื่อมต่อ `trip` มี `alarms_raised` (จำนวนทั้ง `trip`) และ `last_alarm` ({ "type": "HIGH", "utc": ... } หรือ `null`) ด้วย เพื่อให้ server ที่ติดต่อกำลังไม่ได้รู้ว่ามี alarm เกิดและหายไปแล้วระหว่างนั้น

`READ_LOG_CHUNK` ส่งแสดตามทีกล่องเก็บจริง เป็น JSON ละแสด { "seq": 5, "type": 16, "data": "<base64>" } โดย `data` คือแสดตามผังใน `src/record.h` (record format 3: type 16 ทุก record เป็น ROW คอลัมน์ชุดเดียวกับไฟล์ CSV ดูข้อ 6) `next` คือ seq ที่ขอต่อ หรือ `null` เมื่อหมด แอปเก็บ**ไบนารีตามผัง** ไม่ใช้การตีความของกล่อง แอปรุ่นเก่าจึงไม่ทำ record จาก firmware ใหม่หาย แต่ยังไม่ว่าง

แสดแรก (seq 0, `TRIP_START`) มี header ของ `trip` ต่อท้าย 71 ไบนารี (format 5) ซึ่งมี UUID, วัน และเลขรัน ผังอยู่ใน `src/record.h`

5. คำถามที่ทีมแอปต้องตอบ

- รับหรือขอแก้ UUID, รูปแบบ fragment และ JSON
- แตะเพื่อยืนยันตัว (ข้อ 2): ช่อง `key` ในแก็ก และ AUTH ด้วย HMAC-SHA256 กับ nonce ของแต่ละการเชื่อมต่อ
- `STATUS` ควรอ่านได้โดยไม่ต้อง AUTH โหม (มีอุณหภูมิจำแนก)
- record ใน NFC: เก็บ { "sn", "ble", "key" } โหม และ URL สำหรับแบนเนอร์คืออะไร
- แอปรับ `trip_id` (UUID), `trip_date` และ `trip_number` แทน `trip` แบบตัวเลขได้ไหม และจะแสดงให้ผู้ใช้อย่างไร

6. ส่ง trip ขึ้น server จากแอป (ไฟล์ CSV)

เมื่อผู้ใช้เลือก "จบ trip และส่งข้อมูล" แอปดึงทั้ง trip จากกล่อง สร้างไฟล์ CSV แล้วส่งให้ server เอง (ช่องทางรับไฟล์ยังรอตกลงกับทีม server) ภาพรวมอยู่ที่ ../server/trip-data-flow.md

- 1. เรียก STOP_TRIP (ถ้า trip ยังวิ่ง)
- 2. อ่านทั้ง trip ด้วย READ_LOG_CHUNK ที่ละ 16 แถว (trip_id , from , max) จนได้ next เป็น null แต่ละ record เป็น base64 ของแถว 32 ไบต์ ผังไบต์อยู่ใน firmware/src/record.h (READ_LOG_CHUNK เรียกว่า CSV รว 5 แถว)
- 3. แปลงเป็น CSV คอลัมน์ตามตารางด้านล่าง (คอลัมน์สุดท้ายคือไฟล์ที่กล่องทำให้อนโดสพ USB) ชื่อไฟล์ TRIP_<SN>_<YYMMDDhhmm>.csv เช่น TRIP_mCDV1-L0169-1069-002_2610071424.csv (SN เติมตามฉลาก + เวลาท้องถิ่นตอนเริ่ม trip) ขึ้นบรรทัดด้วย CRLF
- 4. ส่งไฟล์ขึ้น server
- 5. server ตอบว่าเก็บแล้ว → เรียก MARK_DELIVERED { "trip_id": "<uuid>" } (ต้อง AUTH) กล่องจะไม่ส่ง trip นี้ทาง Wi-Fi อีก ถ้าส่งไม่สำเร็จ ไม่ต้องเรียก กล่องยังถือว่า "รอส่ง" และส่งทาง Wi-Fi เองภายหลัง

หัวบรรทัดแรก:

```
trip_id,trip_date,trip_number,seq,sn,timestamp,utc,event,temp,tempmin,tempmax,alarm,timeok,gnssstate,latitude,longitude,motion,battery,internet,detail
```

คอลัมน์	ความหมาย
trip_id	UUID ของ trip ที่กล่องสร้าง
trip_date	วันที่ท้องถิ่นตอนเริ่ม trip YYYYMMDD
trip_number	เลขรันของ trip ในวันนี้ เริ่มที่ 1
seq	ลำดับแถวใน trip เริ่มที่ 0 (trip_id, seq) คือคีย์
sn	serial number ของกล่อง (มีใน CSV; ใน MQTT อยู่ที่หัวข้อ)
timestamp	MQTT: วินาที Unix (UTC) เป็นตัวเลข · CSV: เวลาท้องถิ่น hh:mm:ss DD/MM/YYYY (ค่า tz_offset_min ปกติ +07:00)
utc	เฉพาะ CSV: ช่วงเวลาเดียวกันเป็นวินาที Unix
event	แถวนี้คืออะไร (ตารางด้านล่าง)
temp	°C หลังปรับเทียบ; null เมื่อ probe ไม่ให้ค่า
tempmin / tempmax	เกณฑ์ alarm ของ trip ตั้งแต่เริ่ม
alarm	alarm ที่ค้างหลังแถวนี้ HIGH , LOW , PROBE , BATTERY ค้นด้วย ว่าง = ไม่มี
timeok	false เมื่อกล่องไม่รู้เวลา ตอนนั้น timestamp / utc เป็น null และใช้ boot + up_s เรียงลำดับแทน
gnssstate	fix (ภายในรอบบันทึกล่าสุด), last (ตำแหน่งเก่านั้น), none (ยังไม่เคยได้ตำแหน่งตั้งแต่บูต)
latitude / longitude	องศา; null เมื่อ none
motion	จำนวนครั้งที่ขยับตั้งแต่แถวก่อน
battery	แบต %; null เมื่อไม่ทราบ
internet	ผลการส่งรอบล่าสุด: online , wifi_only (มี Wi-Fi แต่ไม่ถึง broker), offline
detail	เฉพาะ CSV: รายละเอียดของ event เป็นคำ (ด้านล่าง) ส่วนใหญ่ว่าง

event	เกิดเมื่อ	detail
TRIP_START / TRIP_STOP	trip เริ่ม / จบ	
SAMPLE	ทุกรอบบันทึก	
ALARM_HIGH / ALARM_LOW	เกิน tempmax / ต่ำกว่า tempmin นานกว่า dwell	
ALARM_PROBE	ไม่มีอุณหภูมิเกิน 1 นาที	
BATTERY_LOW	แบตต่ำกว่า 15%	%
ALARM_CLEAR	alarm หาย	ชนิดที่หาย
ALARM_ACK	มีคนรับทราบ alarm	alarm ที่รับทราบ
PROBE_FAULT / PROBE_OK	probe หยุดตอบ / กลับมาตอบ	สาเหตุ
USB_IN / USB_OUT	เสียบ / ถอดไฟนอก	
POWER_ON	trip ทำต่อหลังรีเซ็ต	สาเหตุของการรีเซ็ต
POWER_OFF	แบตเตอรี่หมด กล่องตัดตัวเอง	mV
TIME_SET	มีการตั้งเวลา	เวลาก่อนตั้ง
DATA_LOST	trip เก่าถูกลบเพราะพื้นที่เต็ม	เลขของ trip นั้น (ของกล่อง)
SHOCK	สำรอง: กระแทกเกินเกณฑ์ (ยังไม่ได้ตั้งเกณฑ์)	

การขยับธรรมดาไม่เป็นแถว นับรวมในคอลัมน์ motion ค่าว่างใน CSV คือ null

ถ้าได้ trip เดียวกันทั้งจากแอปและจาก Wi-Fi ของกล่อง server กรองซ้ำด้วย (trip_id, seq)